

Python Assignment 1

Data Structures - Strings & Tuples

NAME: UMAYAL.K

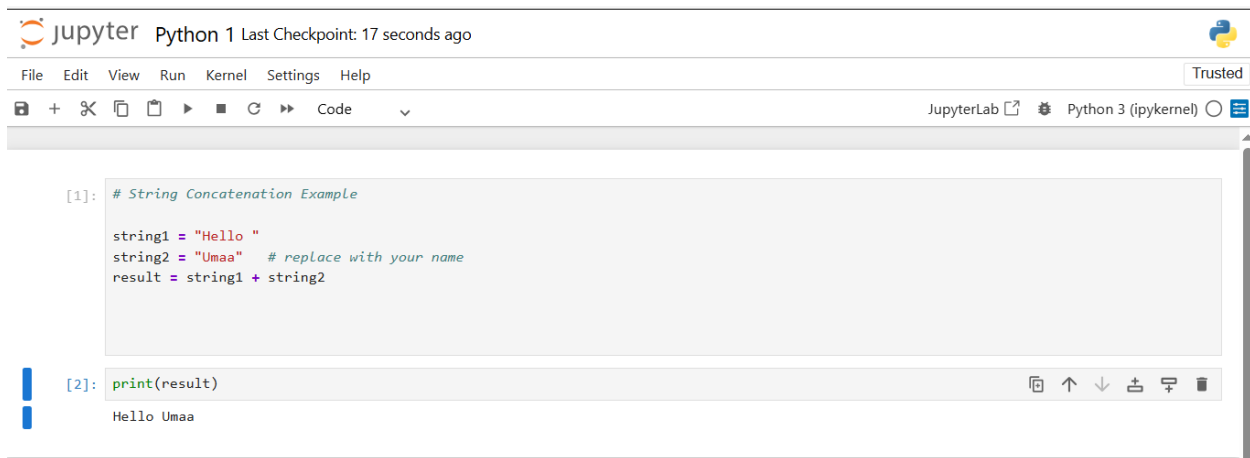
SUBMISSION DATE: 26 - 09 - 2026

PREVIEWS:

Strings:

1. String Concatenation:

string 1 "Hello ",string2 " umaa",string3 "welcome to Python programming"



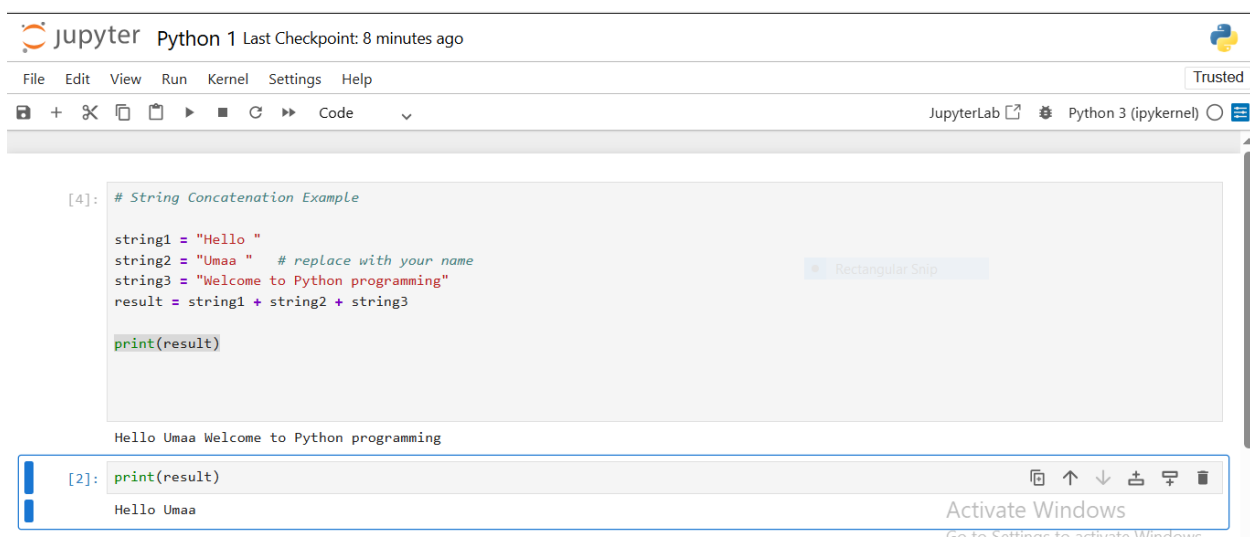
The screenshot shows a JupyterLab interface with a Python 1 kernel. The top bar includes the Jupyter logo, the text "Python 1 Last Checkpoint: 17 seconds ago", and a "Trusted" status indicator. The menu bar contains "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". The toolbar includes icons for saving, adding, deleting, and running code. The code cell contains the following Python code:

```
[1]: # String Concatenation Example

string1 = "Hello "
string2 = "Umaa" # replace with your name
result = string1 + string2

[2]: print(result)

Hello Umaa
```



The screenshot shows a JupyterLab interface with a Python 1 kernel. The top bar includes the Jupyter logo, the text "Python 1 Last Checkpoint: 8 minutes ago", and a "Trusted" status indicator. The menu bar contains "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". The toolbar includes icons for saving, adding, deleting, and running code. The code cell contains the following Python code:

```
[4]: # String Concatenation Example

string1 = "Hello "
string2 = "Umaa" # replace with your name
string3 = "Welcome to Python programming"
result = string1 + string2 + string3

print(result)

Hello Umaa Welcome to Python programming

[2]: print(result)

Hello Umaa
```

An "Activate Windows" watermark is visible in the bottom right corner of the screenshot.

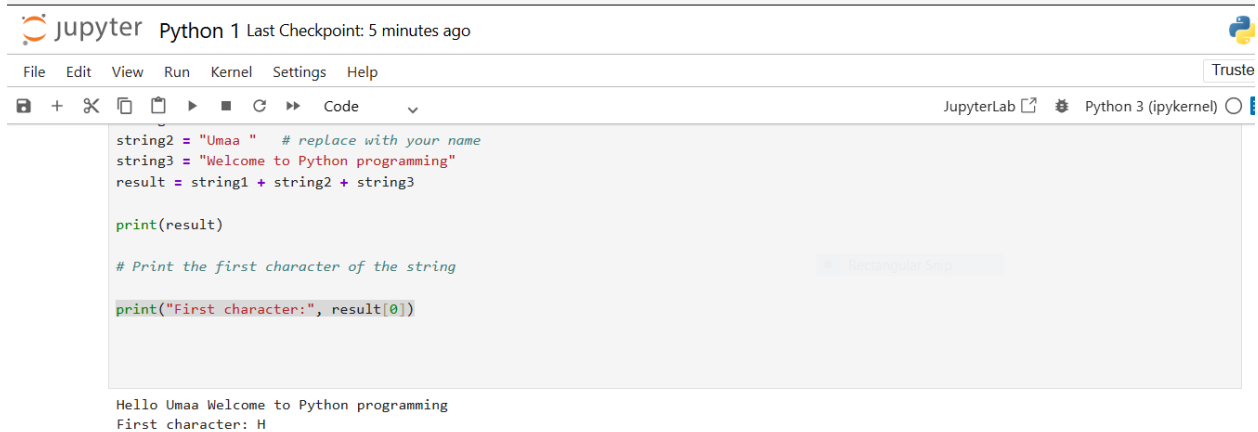
2. String Slicing and Indexing:

A Python program using the above concatenated string as input and performs the following tasks:

a. Print the first character of the string.

“Hello Umaa Welcome to Python Programming”

The first character is “H”



The screenshot shows a JupyterLab interface with a Python 1 kernel. The code in the cell concatenates three strings: 'Hello ', 'Umaa ', and 'Welcome to Python programming'. It then prints the result and the first character of the concatenated string using indexing [0].

```
string1 = "Hello "
string2 = "Umaa " # replace with your name
string3 = "Welcome to Python programming"
result = string1 + string2 + string3

print(result)

# Print the first character of the string

print("First character:", result[0])
```

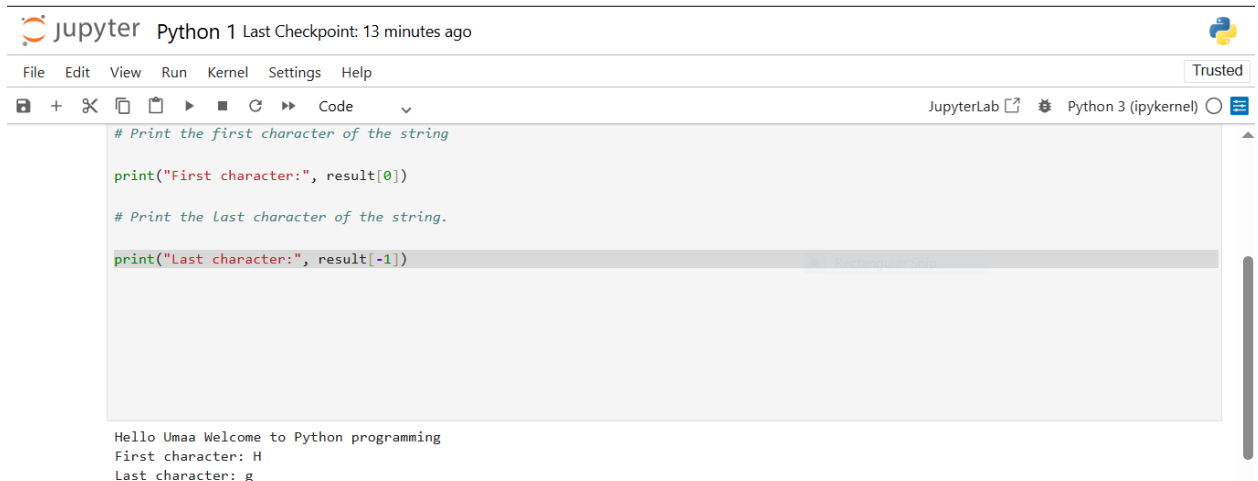
Output:

```
Hello Umaa Welcome to Python programming
First character: H
```

b. Print the last character of the string.

“Hello Umaa Welcome to Python Programming”

The last character is “g”



The screenshot shows a JupyterLab interface with a Python 3 (ipykernel) kernel. The code in the cell concatenates three strings: 'Hello ', 'Umaa ', and 'Welcome to Python programming'. It then prints the result, the first character using indexing [0], and the last character using indexing [-1].

```
# Print the first character of the string

print("First character:", result[0])

# Print the Last character of the string.

print("Last character:", result[-1])
```

Output:

```
Hello Umaa Welcome to Python programming
First character: H
Last character: g
```

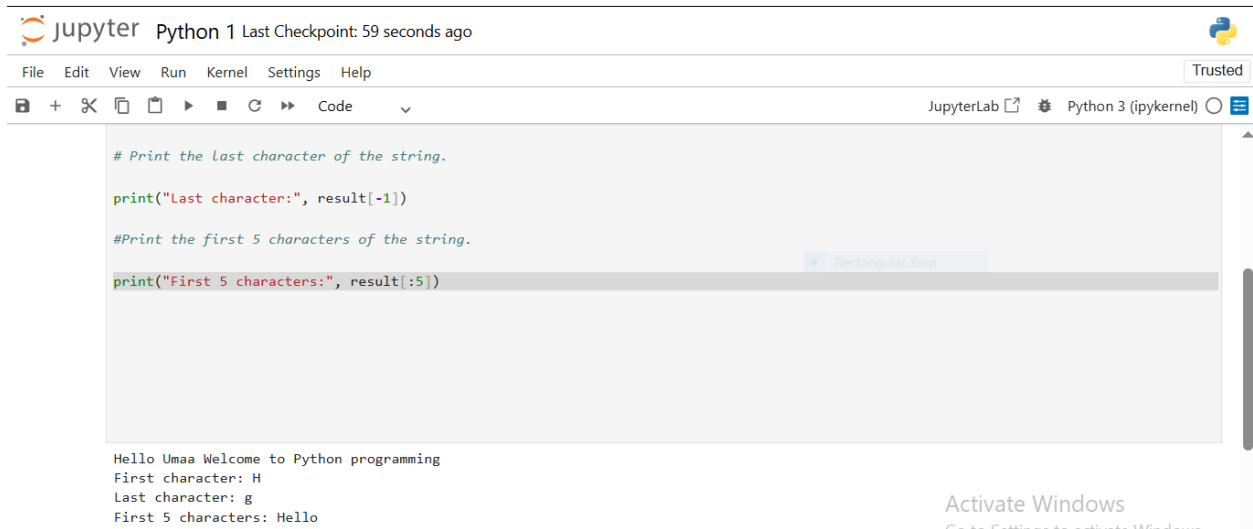
c. Print the first 5 characters of the string.

“Hello Umaa Welcome to Python Programming”

Result:

The first 5 characters is **“Hello”**

- In Python, `result[:5]` means “take everything from the start up to index 5 (not including 5).” That gives you the first 5 characters of the string.



The screenshot shows a JupyterLab window with a Python 1 kernel. The code editor contains the following code:

```
# Print the Last character of the string.  
print("Last character:", result[-1])  
  
#Print the first 5 characters of the string.  
print("First 5 characters:", result[:5])
```

The output area shows the following results:

```
Hello Umaa Welcome to Python programming  
First character: H  
Last character: g  
First 5 characters: Hello
```

There is a watermark "Activate Windows" in the bottom right corner of the screenshot.

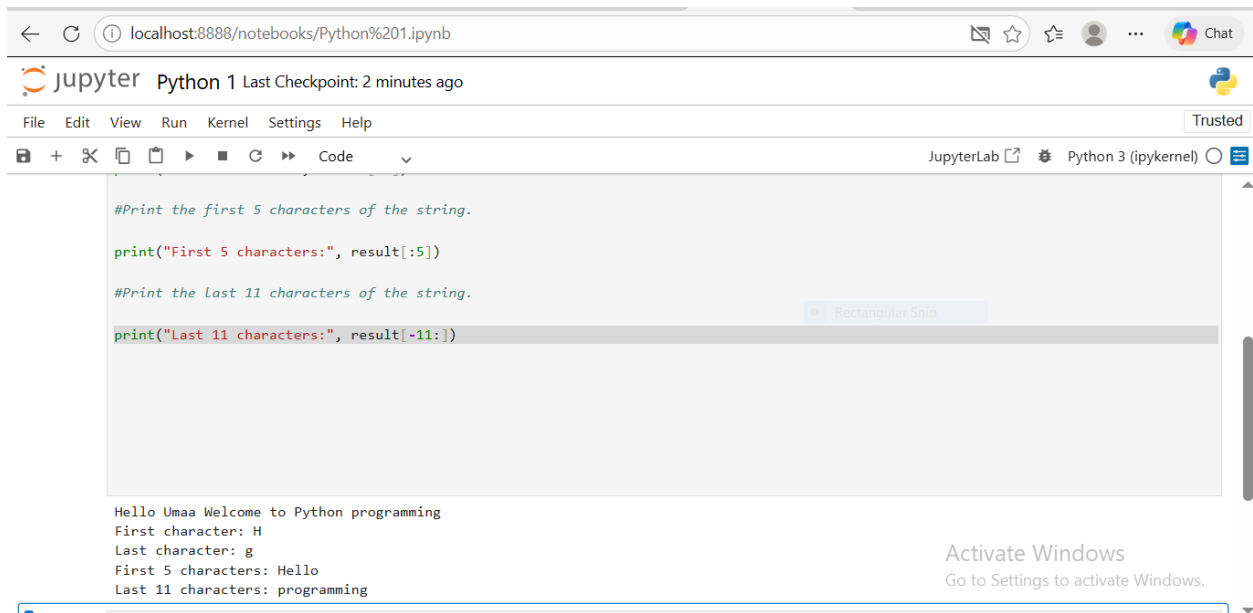
d. Print the last 11 characters of the string.

“Hello Umaa Welcome to Python Programming”

Result:

The last 11 characters is **“Programming”**

- `result[-11:]` means “start 11 characters from the end and go to the end of the string.”



The screenshot shows a JupyterLab window with a Python notebook. The browser address bar indicates the URL is localhost:8888/notebooks/Python%201.ipynb. The notebook title is "Python 1" and it shows the last checkpoint was 2 minutes ago. The code in the cell is as follows:

```
#Print the first 5 characters of the string.  
print("First 5 characters:", result[:5])  
  
#Print the Last 11 characters of the string.  
print("Last 11 characters:", result[-11:])
```

The output of the code is displayed below the code cell:

```
Hello Umaa Welcome to Python programming  
First character: H  
Last character: g  
First 5 characters: Hello  
Last 11 characters: programming
```

An "Activate Windows" watermark is visible in the bottom right corner of the notebook area.

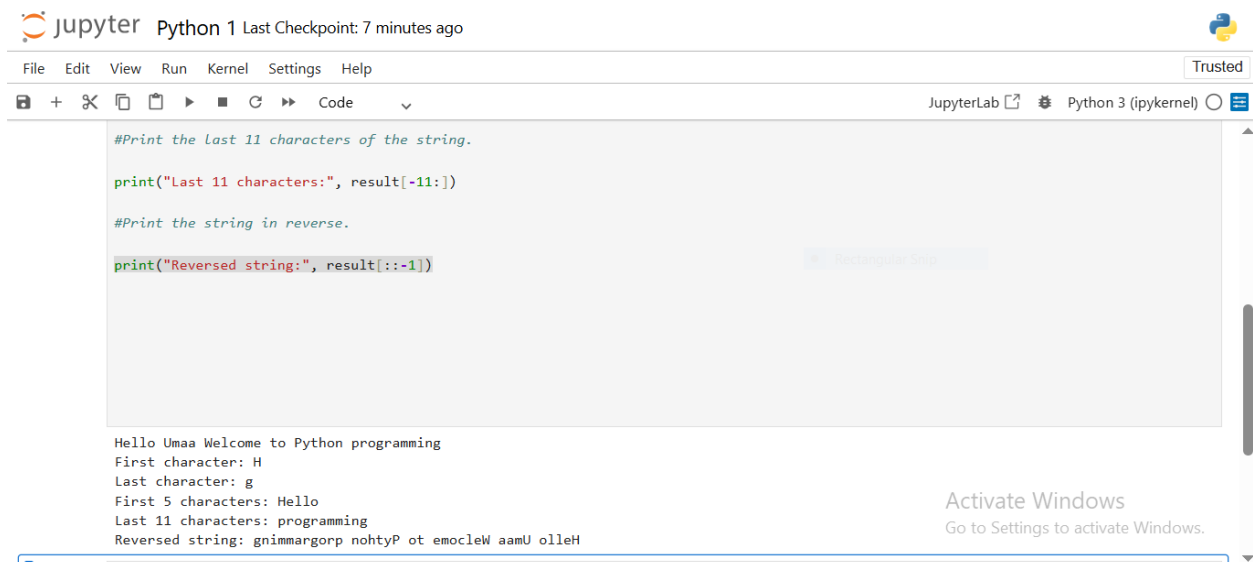
e. Print the string in reverse.

“Hello Umaa Welcome to Python Programming”

Result:

The string in reverse is “gnimmargorp nohtyP ot emoclew,aamU olleH”

- `result[::-1]` tells Python to take the string from start to end but step backwards (`-1`), effectively reversing it.



The screenshot shows a JupyterLab window with a Python notebook. The browser address bar indicates the URL is localhost:8888/notebooks/Python%201.ipynb. The notebook title is "Python 1" and it shows the last checkpoint was 7 minutes ago. The code in the cell is as follows:

```
#Print the Last 11 characters of the string.  
print("Last 11 characters:", result[-11:])  
  
#Print the string in reverse.  
print("Reversed string:", result[::-1])
```

The output of the code is displayed below the code cell:

```
Hello Umaa Welcome to Python programming  
First character: H  
Last character: g  
First 5 characters: Hello  
Last 11 characters: programming  
Reversed string: gnimmargorp nohtyP ot emoclew aamU olleH
```

An "Activate Windows" watermark is visible in the bottom right corner of the notebook area.

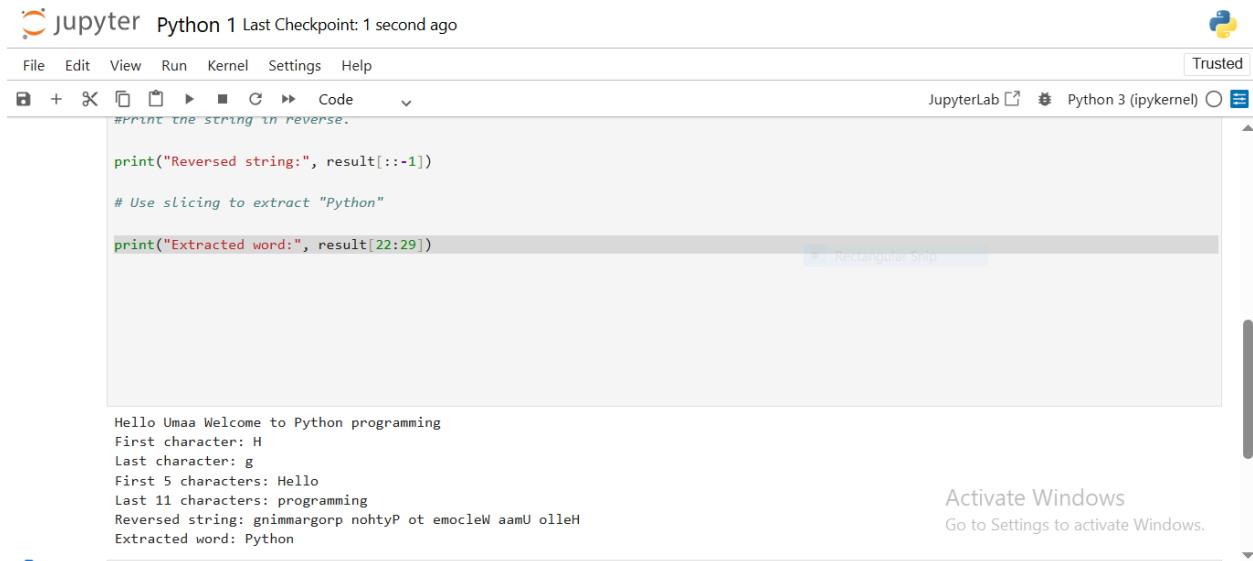
f. Use slicing and print the word “Python” from the existing string.

“Hello Umaa Welcome to Python Programming”

Result:

Extracted word: Python

- `result[22:29]` means “start at index 22 and stop before index 29.”
- That slice corresponds exactly to “Python” in the string.



The screenshot shows a JupyterLab window titled "Python 1 Last Checkpoint: 1 second ago". The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The main area contains a code editor with the following Python code:

```
#Print the string in Reverse.  
  
print("Reversed string:", result[::-1])  
  
# Use slicing to extract "Python"  
  
print("Extracted word:", result[22:29])
```

Below the code editor, the output of the script is displayed:

```
Hello Umaa Welcome to Python programming  
First character: H  
Last character: g  
First 5 characters: Hello  
Last 11 characters: programming  
Reversed string: gnimmargorp nohtyP ot emocleW aamU olleH  
Extracted word: Python
```

An "Activate Windows" watermark is visible in the bottom right corner of the output area.

3. String Methods:

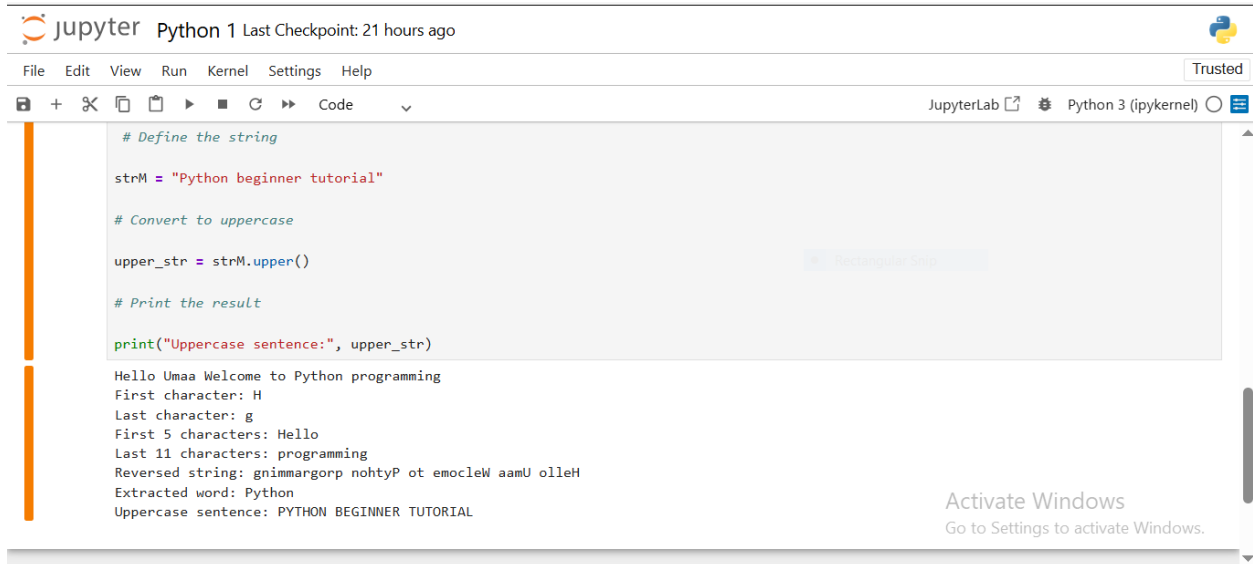
(a).Convert the sentence to uppercase.

Python program that takes a string, strM = “Python beginner tutorial”

Result:

Uppercase sentence: PYTHON BEGINNER TUTORIAL

- To convert a sentence to uppercase in Python, you can use the `.upper()` method.
- The `.upper()` method transforms every character in the string to uppercase.



```
# Define the string
strM = "Python beginner tutorial"

# Convert to uppercase
upper_str = strM.upper()

# Print the result
print("Uppercase sentence:", upper_str)
```

Hello Umaa Welcome to Python programming
First character: H
Last character: g
First 5 characters: Hello
Last 11 characters: programming
Reversed string: gnimmargorp nohtyP ot emocleW aamU olleH
Extracted word: Python
Uppercase sentence: PYTHON BEGINNER TUTORIAL

Activate Windows
Go to Settings to activate Windows.

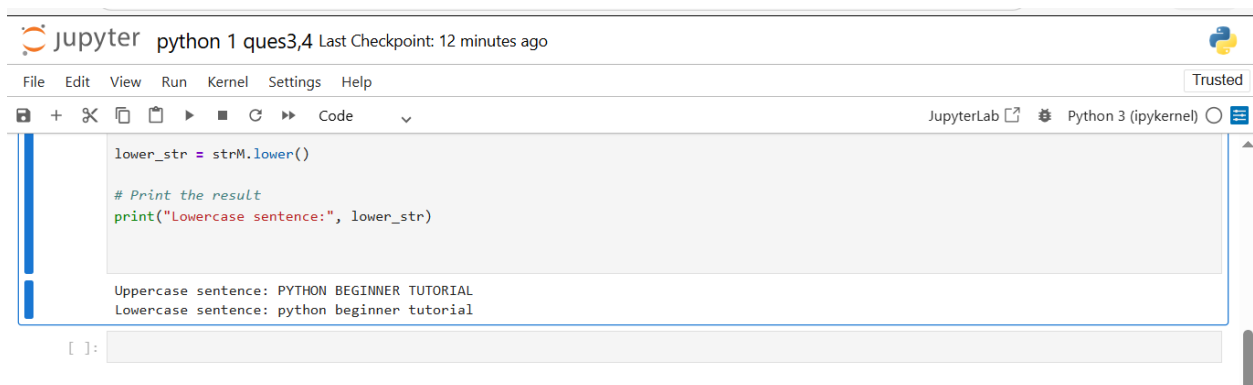
(b).Convert the sentence to lowercase.

Python program that takes a string, strM = “Python beginner tutorial”

Result:

Lowercase sentence: python beginner tutorial

- To convert your sentence to lowercase, you can use Python’s `.lower()` method.
- `.lower()` changes every character in the string to lowercase.



```
lower_str = strM.lower()

# Print the result
print("Lowercase sentence:", lower_str)
```

Uppercase sentence: PYTHON BEGINNER TUTORIAL
Lowercase sentence: python beginner tutorial

[]:

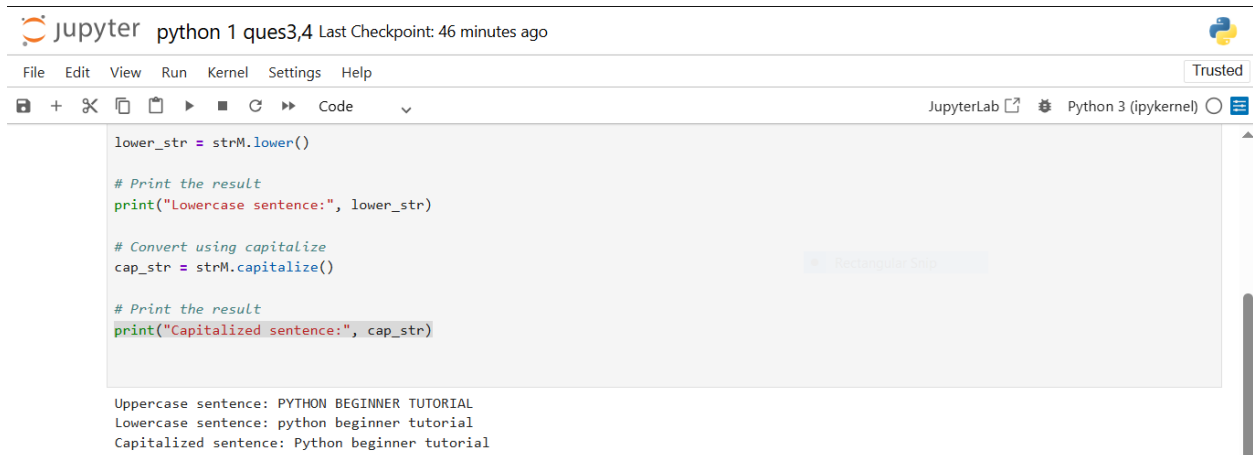
(c) .Use Capitalize and return the sentence to the original input form.

Python program that takes a string, strM = "Python beginner tutorial"

Result:

Capitalized sentence: Python beginner tutorial

- In Python, the `.capitalize()` method converts the first character of the string to uppercase and the rest to lowercase.

A screenshot of a JupyterLab interface. The top bar shows 'jupyter python 1 ques3,4 Last Checkpoint: 46 minutes ago'. The left sidebar has icons for File, Edit, View, Run, Kernel, Settings, and Help. The main area contains a code editor with the following Python code:

```
lower_str = strM.lower()

# Print the result
print("Lowercase sentence:", lower_str)

# Convert using capitalize
cap_str = strM.capitalize()

# Print the result
print("Capitalized sentence:", cap_str)
```

Below the code editor, the output is displayed:

```
Uppercase sentence: PYTHON BEGINNER TUTORIAL
Lowercase sentence: python beginner tutorial
Capitalized sentence: Python beginner tutorial
```

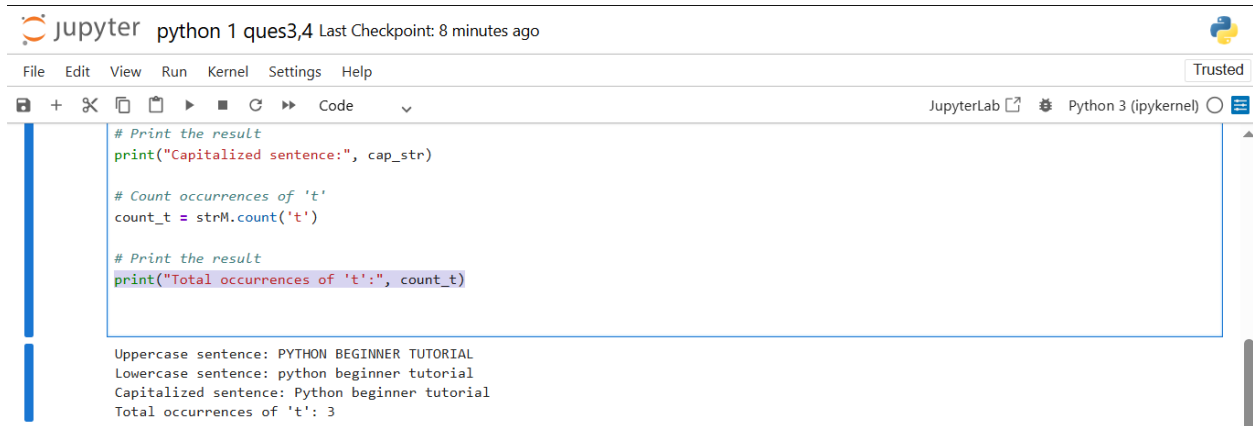
d. Count the total number of occurrences of character 't' in the string.

Python program that takes a string, strM = "Python beginner tutorial"

Result:

Total occurrences of 't': 3

- `.count('t')` scans the entire string and returns how many times 't' appears.
- In "Python beginner tutorial", the character 't' occurs 3 times.



The image shows a JupyterLab window titled "python 1 ques3,4" with a "Last Checkpoint: 8 minutes ago" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor contains the following Python code:

```
# Print the result
print("Capitalized sentence:", cap_str)

# Count occurrences of 't'
count_t = strM.count('t')

# Print the result
print("Total occurrences of 't':", count_t)
```

Below the code editor, the output is displayed:

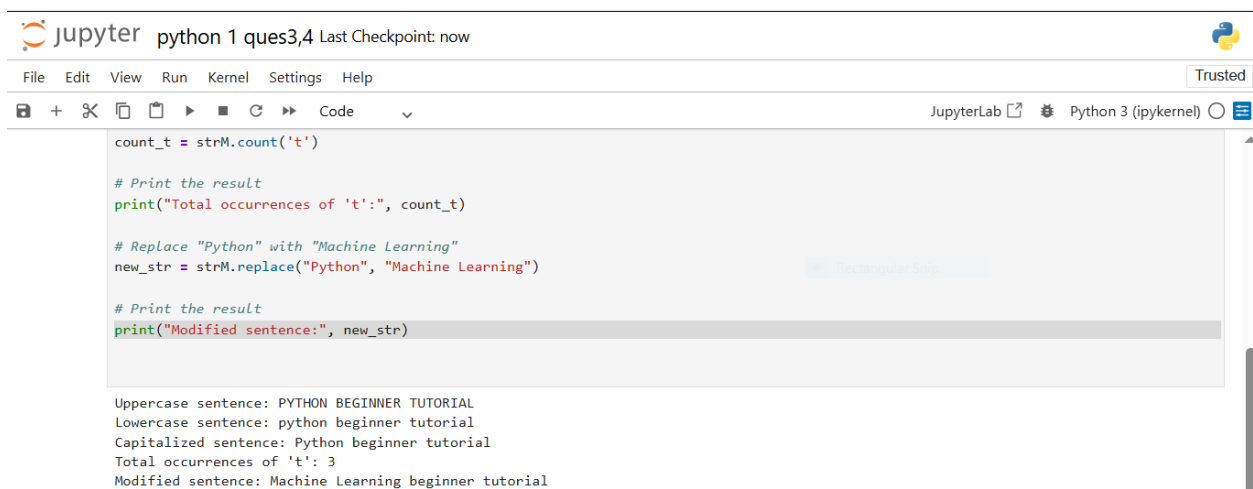
```
Uppercase sentence: PYTHON BEGINNER TUTORIAL
Lowercase sentence: python beginner tutorial
Capitalized sentence: Python beginner tutorial
Total occurrences of 't': 3
```

e. Replace all occurrences of “Python” with “Machine Learning” in the input string strM = “Python beginner tutorial”

Result:

Modified sentence: Machine Learning beginner tutorial

- `.replace("Python", "Machine Learning")` searches for every occurrence of "Python" in the string and replaces it with "Machine Learning".
- If "Python" appeared multiple times, all of them would be replaced.



The image shows a JupyterLab window titled "python 1 ques3,4" with a "Last Checkpoint: now" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor contains the following Python code:

```
count_t = strM.count('t')

# Print the result
print("Total occurrences of 't':", count_t)

# Replace "Python" with "Machine Learning"
new_str = strM.replace("Python", "Machine Learning")

# Print the result
print("Modified sentence:", new_str)
```

Below the code editor, the output is displayed:

```
Uppercase sentence: PYTHON BEGINNER TUTORIAL
Lowercase sentence: python beginner tutorial
Capitalized sentence: Python beginner tutorial
Total occurrences of 't': 3
Modified sentence: Machine Learning beginner tutorial
```

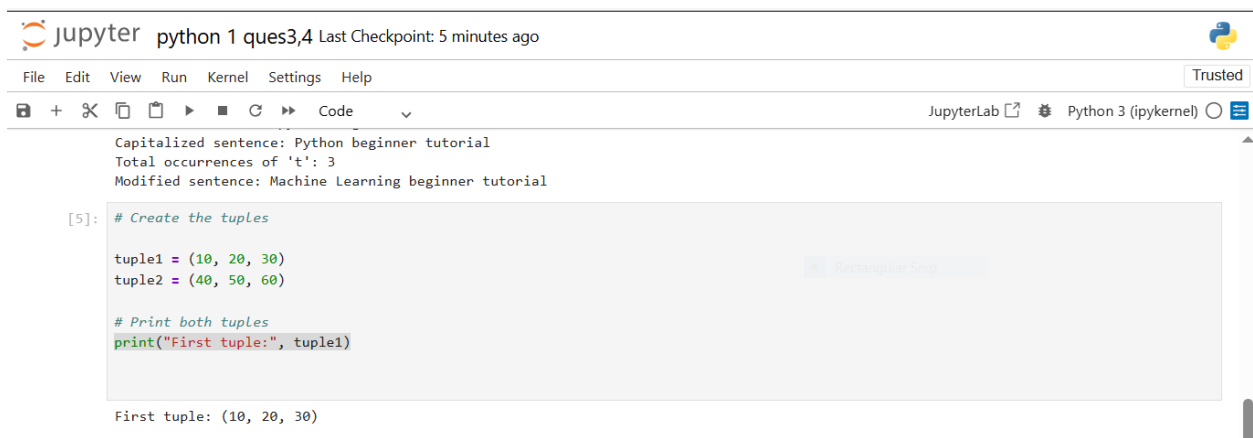
4.Tuples (Creation, Modification and Access) :

Create 1st tuple with values -> (10, 20, 30), 2nd tuple with values -> (40, 50, 60)

Result:

First tuple: (10, 20, 30)

- The **+** operator joins two tuples together.
- Since tuples are immutable, this creates a new tuple **t_combine** without altering the originals.



The screenshot shows a JupyterLab interface with a code cell. The top bar indicates 'python 1 ques3,4' and 'Last Checkpoint: 5 minutes ago'. The code cell contains the following Python code:

```
[5]: # Create the tuples

tuple1 = (10, 20, 30)
tuple2 = (40, 50, 60)

# Print both tuples
print("First tuple:", tuple1)
```

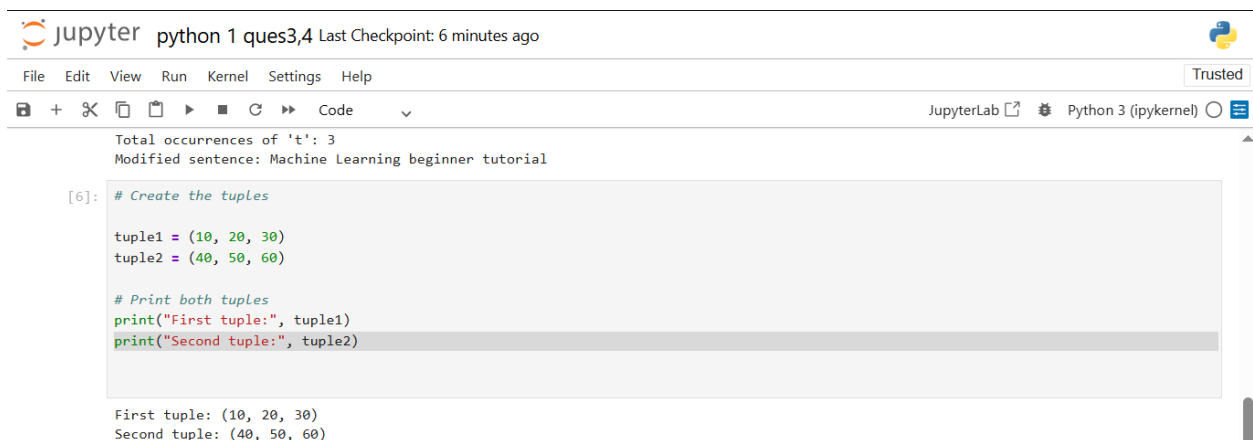
The output of the code cell is:

```
Capitalized sentence: Python beginner tutorial
Total occurrences of 't': 3
Modified sentence: Machine Learning beginner tutorial

First tuple: (10, 20, 30)
```

Result:

Second tuple: (40, 50, 60)



The screenshot shows a JupyterLab interface with a code cell. The top bar indicates 'python 1 ques3,4' and 'Last Checkpoint: 6 minutes ago'. The code cell contains the following Python code:

```
[6]: # Create the tuples

tuple1 = (10, 20, 30)
tuple2 = (40, 50, 60)

# Print both tuples
print("First tuple:", tuple1)
print("Second tuple:", tuple2)
```

The output of the code cell is:

```
Total occurrences of 't': 3
Modified sentence: Machine Learning beginner tutorial

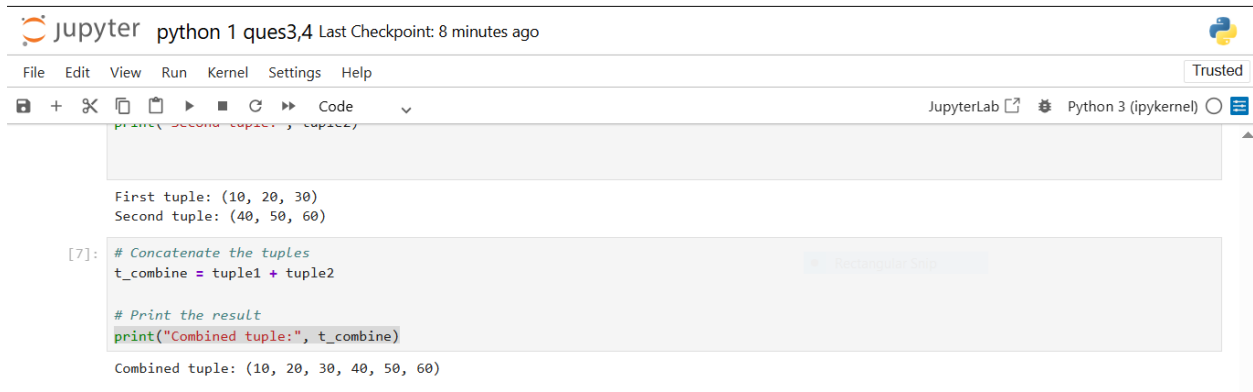
First tuple: (10, 20, 30)
Second tuple: (40, 50, 60)
```

a. Concatenate the two tuples and store it in “t_combine”

Result:

Combined tuple: (10, 20, 30, 40, 50, 60)

- `t_combine = tuple1 + tuple2`
- The `+` operator joins two tuples together.
- Since tuples are immutable, this creates a new tuple `t_combine` without altering the originals.



The image shows a JupyterLab interface with a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main area contains a code cell with the following content:

```
print(second_tuple, tuple2)

First tuple: (10, 20, 30)
Second tuple: (40, 50, 60)

[7]: # Concatenate the tuples
      t_combine = tuple1 + tuple2

      # Print the result
      print("Combined tuple:", t_combine)

      Combined tuple: (10, 20, 30, 40, 50, 60)
```

The output of the code cell shows the first and second tuples, followed by the concatenated result: (10, 20, 30, 40, 50, 60).

b. Repeat the elements of “t_combine” 3 times

Result:

Repeated tuple: (10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60)

- `t_combine * 3` repeats the tuple three times in sequence.
- Tuples are immutable, so this creates a new tuple without changing the original.



The image shows a JupyterLab interface with a code cell containing the following content:

```
[8]: # Repeat the elements of t_combine 3 times
      t_repeat = t_combine * 3

      # Print the result
      print("Repeated tuple:", t_repeat)

      Repeated tuple: (10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60)
```

The output of the code cell shows the repeated tuple: (10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60).

c. Access the 3rd element from “t_combine”

Result:

3rd element from t_combine: 30

- `t_combine[2]` retrieves the third element because indexing starts at 0.
- So, the elements are:
Index 0 → 10
Index 1 → 20
Index 2 → 30

```
[9]: # Access the 3rd element (index 2)
      third_element = t_combine[2]

      # Print the result
      print("3rd element from t_combine:", third_element)

3rd element from t_combine: 30
```

d. Access the first three elements from “t_combine”

Result:

First three elements from t_combine: (10, 20, 30)

- `t_combine[:3]` means “take elements from the start up to index 3 (not including 3).”
- That gives you the first three elements `(10, 20, 30)`.

```
[10]: # Access the first three elements
       first_three = t_combine[:3]

       # Print the result
       print("First three elements from t_combine:", first_three)

First three elements from t_combine: (10, 20, 30)
```

e. Access the last three elements from “t_combine”

Result:

Last three elements from `t_combine`: (40, 50, 60)

- `t_combine[-3:]` means “start 3 elements from the end and go all the way to the end.”
- Since `t_combine = (10, 20, 30, 40, 50, 60)`, the last three elements are (40, 50, 60).

```
[11]: # Access the last three elements
      last_three = t_combine[-3:]

      # Print the result
      print("Last three elements from t_combine:", last_three)
```

Last three elements from t_combine: (40, 50, 60)